Alen Titus

Mechatronics & Robotics Portfolio

B.Tech Mechatronics Engineering (Robotics), SRM IST | Graduating 2027

at8977@srmist.edu.in | +91 7012551196 | Potheri, Tamil Nadu | LinkedIn: www.linkedin.com/in/alen-titus

About me

Fourth-year Mechatronics (specialization in Robotics) student at SRM IST. I build robots that combine mechanical design, electronics and software, and this file shows four projects, what I built, and the problems I solved along the way.

Design tools	SolidWorks, ANSYS Mechanical, ANSYS Icepak
Code and control	C++, Python, Java, ROS2, OpenCV, PID control, embedded systems

Contents at a glance

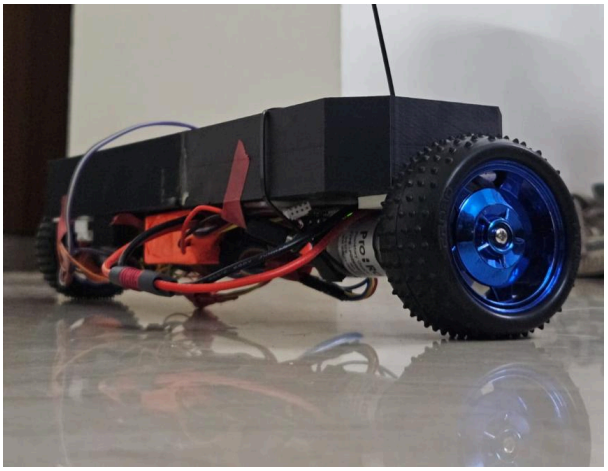
Project	Domain	Key skills shown
Self-Balancing Robot	Control systems, embedded	IMU sensor fusion, PID tuning, motor and encoder interfacing, chassis design
Vision-Guided Pick-and-Place	Robotics, computer vision	OpenCV detection, calibration, ArUco, Dobot control, stepper integration
Gesture-Based Pneumatic Control System	Fluid power, vision	Hand tracking, PyFirmata, solenoid valve control
Dual-Arm Magnetic Pick-and-Place	Mechatronics	Stepper motion control, homing, electromagnetic gripper
Internship at RFAS	Industrial automation	Exposure to automated machines and control panels in a factory

1. Self-Balancing Two-Wheeled Robot

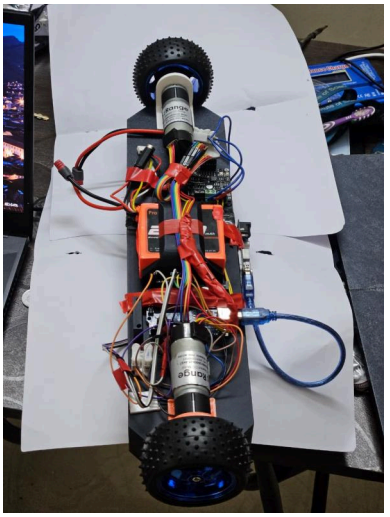
Self-Balancing Two-Wheeled Robot

Jan–May 2026 | College minor project, team of three | Arduino, C++, PID control

What it is. A two-wheeled robot that keeps itself upright, like an inverted pendulum. An IMU measures the tilt, an Arduino Uno works out the correction with a PID controller, and two geared motors drive the wheels under the body to catch the fall. Encoders on both motors give feedback on wheel movement.



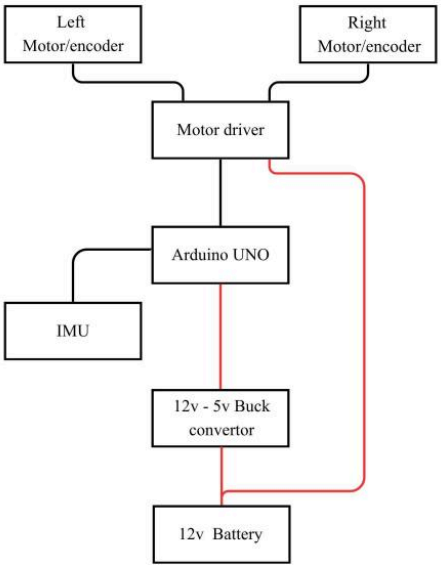
The assembled robot



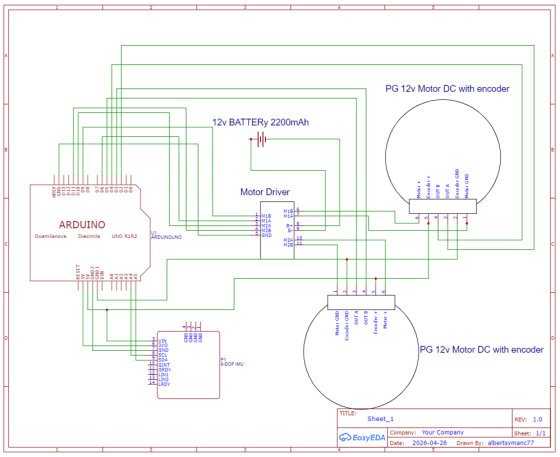
Wiring and component layout

System and hardware

Controller	Arduino Uno (ATmega328P, 16 MHz)
Tilt sensing	MPU6500 IMU (gyroscope and accelerometer) over I2C
Actuation	12 V planetary gear DC motors with built-in quadrature encoders
Motor driver	Cytron dual-channel driver, PWM speed and direction
Power	12 V 2200 mAh battery with a 12 V to 5 V buck converter for the logic side
Body	Custom chassis and wheel brackets, modelled in CAD (views below)



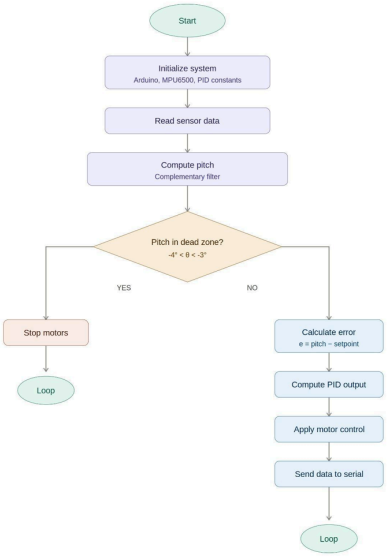
System block diagram



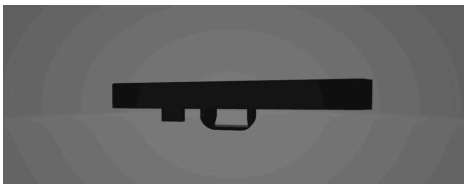
Circuit schematic

How the control works

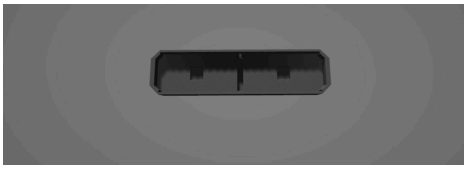
- Tilt angle: the accelerometer is accurate over time but noisy, and the gyroscope is smooth but drifts. A complementary filter (alpha = 0.98) blends the two into one stable pitch angle.
- Control loop: the controller compares pitch with the setpoint, computes a PID output, and maps it to PWM on both motors. Output is limited to the valid PWM range, and the integral term is clamped to stop windup.
- Dead band: a small angle band around the setpoint stops the motors so the robot does not chatter when it is almost upright.
- Encoders: pulses are counted with interrupts to track wheel motion and support the closed loop.



Control loop flowchart



Chassis, side view (CAD)



Chassis, top view (CAD)



Wheel bracket (CAD)

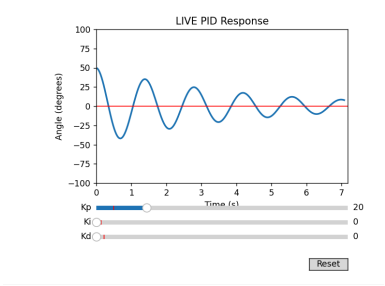
Problems I ran into and how they were solved

Problem	What was done
Motors did not move at low PWM (dead zone)	Set a minimum PWM threshold so small corrections still move the wheels
Tilt reading drifted and was noisy	Fused gyroscope and accelerometer data with a complementary filter
Weight distribution made it unstable	Repositioned the battery, controller and sensors on the chassis
Encoder signals picked up electrical noise	Added pull-up resistors and improved grounding
Left and right motors responded differently	Calibrated both motors to match their response

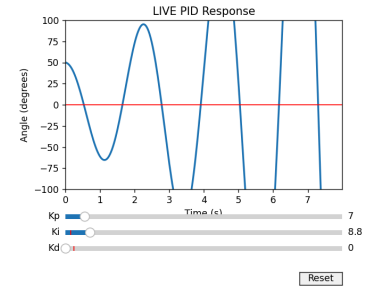
PID tuning trials

I tuned the gains by trial and error, first one term at a time, then in combination. The table shows all eleven trials.

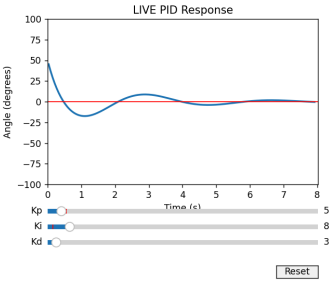
Trial	Kp	Ki	Kd	Observation
1	10	0	0	Continuous oscillation, does not settle
2	20	0	0	Faster, but oscillations grow
3	45	0	0	Very large oscillations, cannot hold balance
4	0	6.7	0	Very slow, drifts
5	0	15.6	0	Overshoot and instability from integral build-up
6	0	31	0	Diverges rapidly
7	0	0	5.2	Small damping effect, does not correct the error
8	0	0	10	Slightly better damping, still misses the target
9	0	0	48.8	Too sluggish from excessive damping
10	7	8.8	0	Less steady-state error, but still oscillates
11	5	8	3	Stable, minimal oscillation, low overshoot, quick settling



Trial 2 (Kp 20): oscillation



Trial 10 (Kp 7, Ki 8.8): oscillation



Trial 11 (Kp 5, Ki 8, Kd 3): best

Outcome and next steps

- Final gains Kp = 5, Ki = 8, Kd = 3 gave the most stable behaviour of the trials, and the robot held its balance and recovered from disturbances.
- Next steps I identified: Kalman filtering or LQR for better accuracy, wireless monitoring with an ESP32, and a load cell so the motors shut off if a rider or load is suddenly removed.

My Contribution

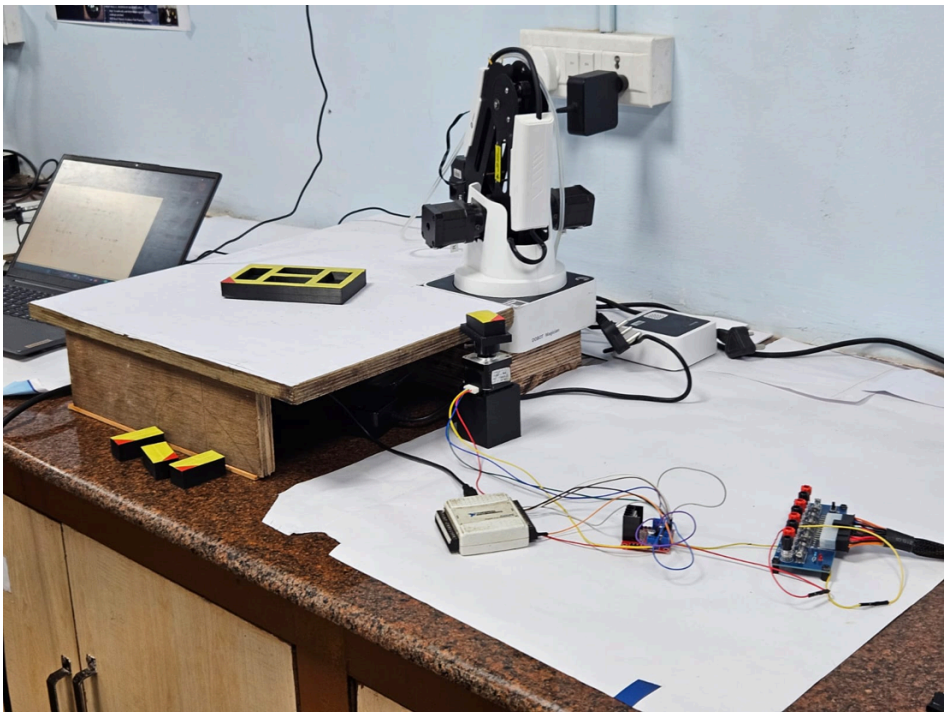
- Writing or tuning the PID controller
- Designing or modelling the chassis and wheel brackets
- Mounting components and balancing the weight

2. Vision-Guided Pick-and-Place System

Vision-Guided Pick-and-Place System

Jan–May 2026 | College team project, Vision Guided Robotics | Python, OpenCV, Dobot Magician

What it is. A system where a camera finds yellow blocks and a slotted pallet, works out where each block is and how it is turned, and a Dobot Magician picks and places it into the right slot. The Dobot cannot rotate the block itself, so a separate stepper mechanism turns each block to match the slot before placement.

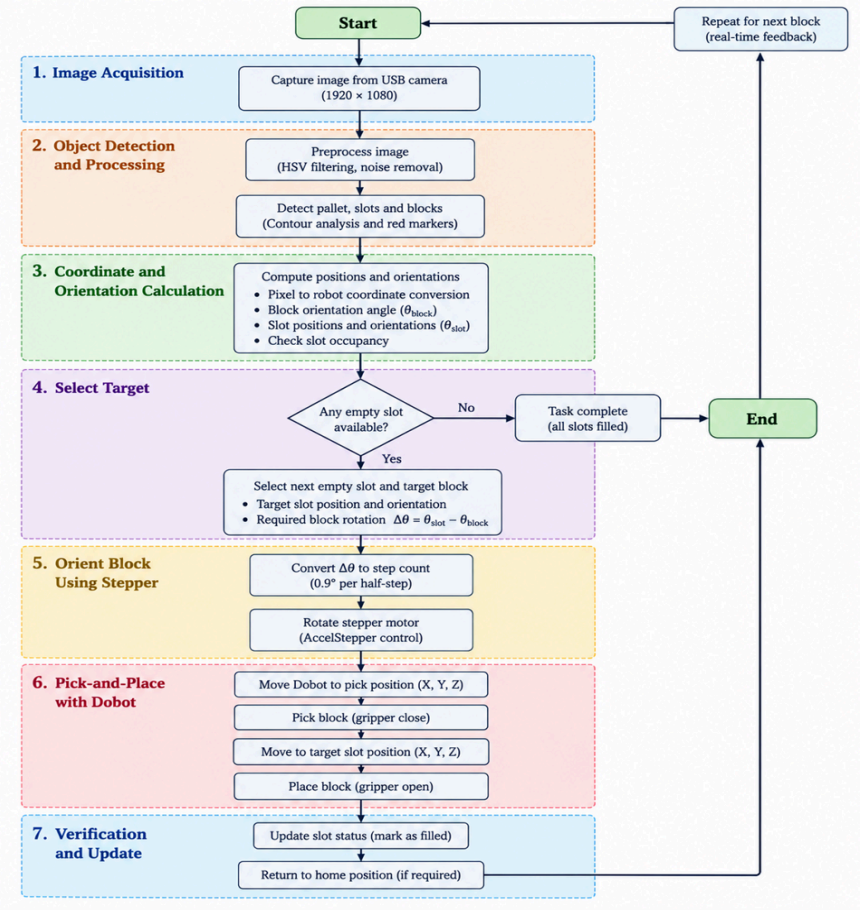


The setup: Dobot Magician, slotted pallet, blocks, stepper rotation mechanism and NI DAQ

How the system works

Stage	What happens
Detect	A USB camera captures 1920 x 1080 images. The image is cleaned up with HSV colour thresholds and morphological filtering. Blocks, red markers and the pallet are found from contours, and minAreaRect gives each block's position and angle.
Calibrate	Pixels are converted to millimetres using a known physical distance. An ArUco marker (ID 1) is the reference origin, with offsets to line the camera frame up with the robot base.
Locate slots	Slot centres are calculated from the pallet centre, its orientation and the known slot spacing. The slots measure 50 x 25 mm.
Check slots	A small region inside each slot is tested for yellow pixels to tell empty slots from filled ones.
Orient	The difference between block angle and slot angle gives the rotation needed. The stepper turns the block in 0.9 degree half-steps using an eight-state coil sequence.
Move	Vision coordinates are mapped to Dobot X/Y and the arm picks and places the block, then the slot is marked as filled.

Program flowchart



Full workflow from image capture to slot update

Problems I ran into and how they were handled

Area	Problem	Approach
Camera	Unstable images and lighting changes	Set manual focus and exposure, and tuned HSV thresholds for red markers, yellow blocks and the pallet
Detection	Noise breaking up contours; blocks confused with the pallet	Morphological opening and closing, plus contour size checks to tell blocks from the pallet
Orientation	A rectangle's angle is ambiguous by 180 degrees	Used the red marker on each block to resolve which way it faces
Calibration	Camera axes did not match the robot axes	Worked out the X/Y mapping and added ArUco-based offsets to align the two frames
Robot accuracy	Dobot moved inconsistently to the same coordinates	Investigated homing, pose readings and how the controller turns Cartesian points into joint moves
Pallet geometry	Sign errors in slot positions; S1/S2 sit 90 degrees from S3/S4	Corrected the direction conventions and modelled the real slot dimensions
Occupancy	Red markers on blocks interfered with pallet detection	Excluded slot regions from orientation detection and tuned the yellow-pixel ratio
Stepper	Turning angle into clockwise or anticlockwise steps	Mapped signed angles to step direction and converted degrees to half-steps
Integration	Missing pydobot, COM port permissions, NumPy 1.x and 2.x conflicts	Resolved the package and serial issues and organised the code into one workflow

My Contribution

- Writing the OpenCV detection (HSV thresholds, contours, minAreaRect)
- Slot geometry and occupancy detection
- Stepper rotation code and angle-to-step conversion
- Dobot control through Spyder and Pydobot
- Building the pallet, blocks or stepper mechanism

The hardest part was making camera, calculations, robot motion and stepper rotation work together in one sequence, because each had to agree on the same coordinates and angles.

Stack	Python, OpenCV, NumPy, NI-DAQmx, PyDobot, ArUco markers
Hardware	Dobot Magician, stepper-driven rotation mechanism, camera, slotted pallet

3. Gesture-Based Pneumatic Control System

Gesture-Based Pneumatic Control System

Jun–Nov 2024 | College team project, Fluid Power Systems | Python, OpenCV, CVZone, Arduino, PyFirmata

What it is. A computer vision system that uses real-time hand gesture recognition to control a pneumatic cylinder. Python handles the vision processing and an Arduino handles the hardware, so a raised hand can move an actuator without touching a switch.



Testing the system on the pneumatic trainer board



Double-acting cylinder and solenoid valve

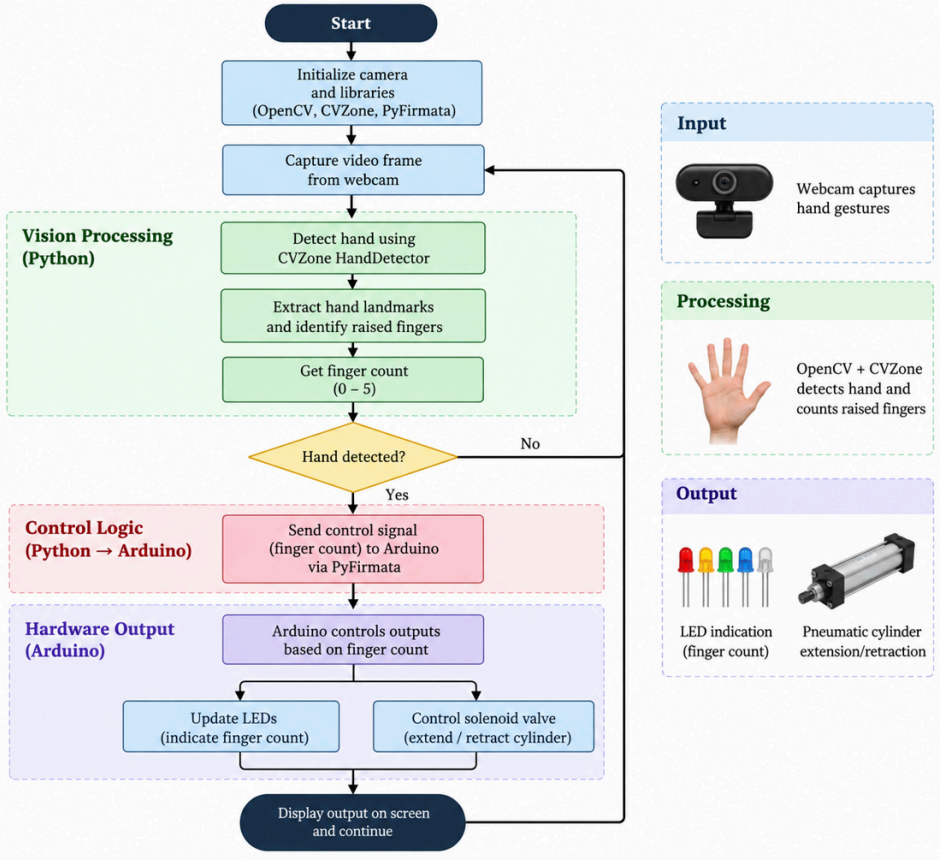
Key features

- Hand gesture recognition: detects hand gestures in real time from a webcam.
- Computer vision: OpenCV and CVZone find the hand landmarks and work out which fingers are raised.
- Pneumatic actuation: drives a double-acting pneumatic cylinder through a solenoid-operated directional control valve.
- Hardware integration: Python talks to the Arduino through PyFirmata.
- Visual feedback: LEDs show the detected finger count and the system status.

Working principle

The webcam captures the hand and the vision algorithm counts the raised fingers. Python sends the matching control signal to the Arduino, which switches the LEDs and a relay. The relay powers the solenoid valve, and the valve sends air to one side of the cylinder or the other.

Flowchart



System flowchart: webcam input, vision processing, Arduino output

My Contribution

- Gesture detection with OpenCV and CVZone
- The Python to Arduino link through PyFirmata

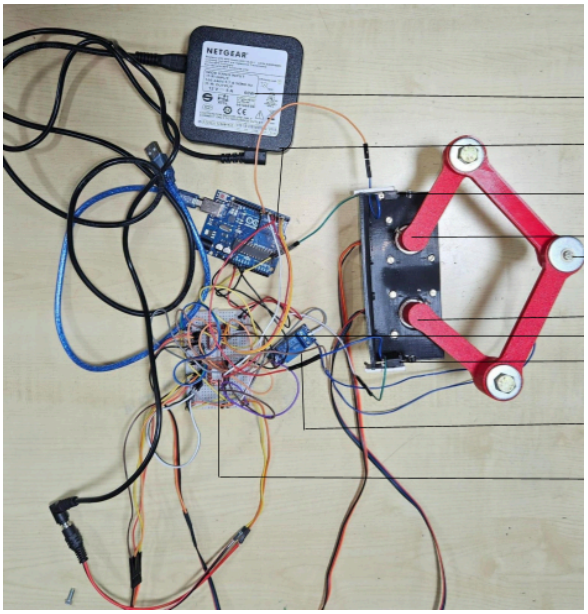
Objective	A simple, contactless human-machine interface for controlling pneumatic motion with hand gestures
Software	Python, OpenCV, CVZone, Arduino IDE, PyFirmata
Hardware	Arduino Uno, relay module, solenoid-operated directional control valve, double-acting pneumatic cylinder, LEDs

4. Dual-Arm Magnetic Pick-and-Place Robot

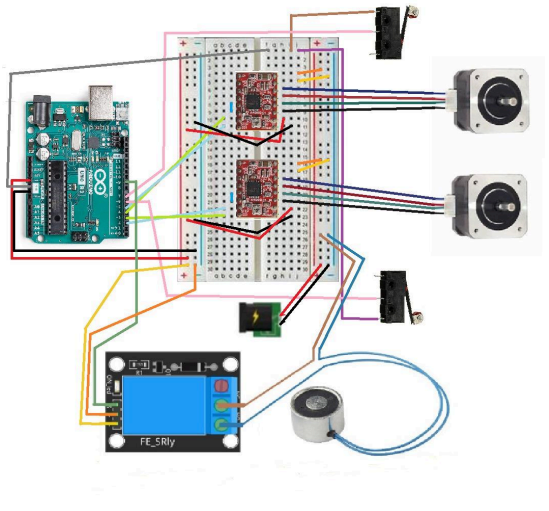
Dual-Arm Magnetic Pick-and-Place Robot

Aug–Nov 2023 | College Team Project, Elements of Mechatronics Systems | C++, Arduino, AccelStepper

What it is. A stepper-driven robot that picks up ferrous objects with an electromagnetic gripper and places them at a position set by the operator. Two NEMA 17 steppers move the arm links, and a relay switches the electromagnet on to pick and off to release, so no mechanical fingers are needed.



The assembled robot with Arduino, drivers and relay



Circuit diagram from the report

Hardware

Controller	Arduino Uno (ATmega328P), programmed in the Arduino IDE with C++
Motion	Two NEMA 17 stepper motors, one per axis (X and Y)
Drivers	Two A4988 stepper drivers with adjustable current limit and microstepping
Gripper	12 V electromagnet switched by a single-channel relay module
Homing	Two limit switches, one per axis
Power and wiring	12 V DC supply, 5 V from the Arduino for logic, breadboard wiring

How it works

- Motion control: the AccelStepper library drives each motor with acceleration and speed limits (maximum 1000 steps per second, acceleration 1000), at 16 microsteps on a 200-step motor.
- Homing: on start-up each axis moves until its limit switch triggers, and that position is set as zero. This gives the arm a known starting point every time.
- Commands: the Arduino reads X and Y target angles over serial, converts them to motor positions and moves both axes together.
- Gripper: a serial command switches the relay, which turns the electromagnet on to hold the object and off to release it.

Pin map

Function	Step	Direction	Limit switch
X axis motor	Pin 2	Pin 3	Pin 4
Y axis motor	Pin 5	Pin 6	Pin 7
Gripper relay	Pin 8 (output)		

Outcome and next steps

- The report records accurate positioning and gripping of ferrous objects in the pick-and-place operation.
- Next steps I identified: gripping non-ferrous objects with vacuum suction, adding depth cameras or force sensors, and tighter integration with vision.

My Contribution

- Wiring the steppers, drivers, limit switches and relay
- Writing the AccelStepper and homing code
- Testing the electromagnet pick and release

5. Internship at Robotica FA Solutions (RFAS)

Internship, Robotics and Industrial Automation

18 June – 15 July 2025 | Robotica FA Solutions (RFAS), Coimbatore, Tamil Nadu

A four-week internship at an industrial automation company. I worked with robotics and automated machines, learned how they operate and where they are used, and saw how automation work runs in a real factory.

- Learned the working principles and applications of automated systems.
- Saw automated machines built with pneumatic cylinders, linear guides, conveyors and motors.
- Saw how control panels are wired, with labelled input and output terminals connected to a PLC.
- Became familiar with industrial automation practices and the workplace environment.



An automated machine with pneumatic grippers, guides and conveyor



Control panel wiring with labelled I/O terminals



Machines under assembly on the factory floor

Certificate of completion


RFAS
Robotica FA Solutions

Date : 15.07.2025

Certificate of Completion

We hereby certify that

Alen Titus

Student of SRM Institute of Science and Technology Chennai has completed internship in the area of Robotics organized by Robotica FA Solutions from 18.06.2025 TO 15.07.2025

We wish him/her all success in future.

Robotica FA Solutions

Govindaraju.A

Director- Operations

Registered Address : C102, Cheran Maa Nagar, Vilankurichi (Po), Coimbatore - 641 035.
Factory : 142/3-1, Bharathi Street, Udayampalayam Main Road, Chinnavedampatti Post, Coimbatore - 641 049.
www.rfas.in

Certificate issued by Robotica FA Solutions, 15 July 2025

6. Skills and Education

Skills

Robotics and control	Embedded systems and microcontrollers, robotic systems, ROS2, PID control, sensor fusion
Vision	OpenCV, colour and contour detection, camera calibration, ArUco markers
Design and simulation	SolidWorks, ANSYS Mechanical, ANSYS Icepak, mechanical design
Programming	C++, Python, Java
Languages	English (professional), Malayalam (native), Hindi and Tamil (limited working)

Education

2027	B.Tech Mechatronics Engineering (Robotics), SRM IST Kattankulathur. CGPA 7.29/10
2023	Class XII (ISC), Hari Sri Vidya Nidhi School, Kerala. 75.8%
2021	Class X (ICSE), Hari Sri Vidya Nidhi School, Kerala. 75.3%

Contact: at8977@srmist.edu.in | +91 7012551196 | Potheri, Tamil Nadu